MANUAL TÉCNICO DEL PROGRAMADOR Y ARQUITECTURA DE CÓDIGO

Proyecto: Juego de Naves en Consola con POO (Actividad 8)

Institución: E.E.S.T. N°1 — Vicente López

Asignatura: Laboratorio de Programación (LPR) — 5° Año

Docente: Mansilla Muñoz York Elías

Autor / Grupo: León Melgarejo

Fecha: 21 de octubre de 2026 | **Versión:** v1.0.0

CONTROL DE VERSIONES Y CHANGELOG

Versión	Fecha	Autor	Descripción de Cambios
v1.0.0	21/10/2026	León Melgarejo	Versión inicial del motor de juego basado en clases e implementación del Sistema de Escudo de Energía avanzado de forma Dual-Remote.

1. ARQUITECTURA DEL SISTEMA Y ENTORNO DE DESARROLLO

Requisitos del Entorno

Compilador: g++ (MinGW-w64 GCC v8.1.0 o superior). Estándar C++11 habilitado.

IDE recomendado: Visual Studio Code con extensión *C/C++ Extension Pack*.

Shell: Windows PowerShell 5.1 o PowerShell Core 7+.

Estructura de Directorios del Repositorio

text

```
juegodenaves/
├── .gitattributes                <-- Normalización de finales de línea
├── .gitignore                    <-- Exclusiones de Git (.exe, .vscode/, *.out)
```

```
├── LICENSE                          <-- Licencia MIT de uso
escolar
├── README.md                        <-- Portada técnica del
proyecto
├── docs/                            <-- Documentación y manuales
técnicos
|   ├── InformeEEST1_LPR2026_ACT08_G03_v1.0.0.pdf <-- Informe
académico APA v7
|   ├── CHANGELOG.md                 <-- Bitácora de versionado
SemVer
|   └── manuales/
|       ├── manual_programador_v1.0.0.pdf
|       ├── manual_programador_v1.0.0.md
|       ├── manual_usuario_v1.0.0.pdf
|       └── manual_usuario_v1.0.0.md
├── src/                             <-- Código fuente compilable
|   └── main.cpp                     <-- Código fuente C++
unificado (Clases y main)
|   └── juegodenaves.exe             <-- Ejecutable local (ignorado
por Git)
└── capturas/                        <-- Evidencias de ejecución
(.png)
    ├── combate_consola.png          <-- Captura de la terminal con
piloto y juego
    └── traza_memoria.png            <-- Diagrama de asignación de
objetos en la RAM
```

2. DOCUMENTACIÓN DE MÓDULOS Y CLASES (src/main.cpp)

Módulo 1: class Nave (Entidad Jugador)

Propósito: Encapsula el estado posicional, las vidas y el blindaje de la nave del usuario, proveyendo métodos controlados para su renderizado y locomoción en la cuadrícula de la consola.

Atributos Privados: `int x`, `int y` (coordenadas cartesianas de pantalla), `int vidas` (cuantificador de estado vital) e `int escudo` (atributo del desafío para absorción de daños).

Métodos Públicos Importantes:

- `Nave(int _x, int _y, int _vidas)`: Constructor encargado de inicializar el estado nativo de la instancia en memoria evitando basura en la RAM.
- `void mover(char tecla)`: Evalúa el caracter ingresado y modifica de forma condicionada las coordenadas geométricas limitando el movimiento dentro del marco perimetral simulado (`x > 2 && x < 70`).
- `void recibirImpacto()`: Algoritmo de mitigación de daños. Si `escudo > 0` descuenta blindaje en intervalos de 25 unidades; si se vacía por completo, devalúa directamente el contador `vidas`.

Módulo 2: class Asteroide (Entidad Obstáculo Dinámico)

Propósito: Gestionar el ciclo de vida y desplazamiento vertical automatizado de los elementos hostiles.

Mecánica de Movimiento (`void mover()`): Incrementa progresivamente la ordenada `y++`. Incorpora una rutina de reciclaje dinámico en memoria: cuando el objeto supera el límite inferior de la pantalla (`y > 23`), limpia su rastro anterior, se reposiciona en el tope superior (`y = 4`) y genera una nueva abscisa pseudoaleatoria mediante la expresión `(rand() % 70) + 2`.

Módulo 3: Interacciones y Colisiones (Bloque en main)

Propósito: Sincronizar el bucle principal de juego (*Game Loop*) mediante retrasos controlados por hardware (`Sleep(30)`) y evaluar interacciones vectoriales.

Lógica de Detección de Colisiones: Implementa una evaluación geométrica comparativa bidimensional dentro de bucles anidados (`for`):

1. *Bala vs Asteroide*: Si las posiciones de los punteros `Bala*` y `Asteroide*` coinciden en el plano cartesiano, se desasigna el proyectil de la memoria física mediante la palabra clave `delete`, se remueve la referencia del contenedor dinámico `std::vector::erase()` y se reasigna el asteroide al origen superior.

3. INSTRUCCIONES DE COMPILACIÓN Y DEPURACIÓN

Compilación desde PowerShell

Ubicándose en la raíz del proyecto (`juegodenaves/`), ejecute secuencialmente:

powershell

```
# Compilación del archivo fuente C++ utilizando el estándar C++11
g++ src/main.cpp -o src/juegodenaves.exe

# Ejecución del binario resultante en la consola de Windows
.\src\juegodenaves.exe
```

Usa el código con precaución.

Diagnóstico de Errores Frecuentes

1. **Error:** `'g++' no se reconoce como un comando interno o externo...`
    - *Causa:* Las variables de entorno globales del sistema operativo Windows (`PATH`) no incluyen la ruta binaria del compilador de MinGW.
    - *Solución:* Incorporar manualmente `C:\mingw64\bin` (o la ruta correspondiente de instalación) a las variables de entorno del sistema y reiniciar la terminal de comandos.
2. **Error:** `fatal error: conio.h: No such file or directory`
    - *Causa:* Intento de compilación en sistemas operativos no basados en la API nativa de Windows (como GNU/Linux o macOS), donde la biblioteca clásica de MS-DOS `conio.h` y sus funciones de captura por buffer como `_kbhit()` o `_getch()` no están disponibles estructuralmente.
    - *Solución:* Compilar nativamente bajo entornos Windows utilizando MinGW-w64.

4. REFERENCIAS

Kernighan, B. W., & Ritchie, D. M. (1988). *El lenguaje de programación C* (2.ª ed.). Prentice Hall.

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.